

Artifact-Bound Non-Vacuous Single-Path Refinement of FreeRTOS V6.1.1 vTaskDelay(2)

Mathematical progress report - blind Isabelle/HOL reconstruction

Status date: 2 August 2026 | Result: fixed P2 witness checked; universal functional correctness remains open

Draft provenance. ChatGPT Pro reviewed the proof design and report wording. Every mathematical claim below was rechecked against the local Isabelle theories, artifact ledger, and final run records. ChatGPT Pro supplied no theorem object and is outside the trusted proof chain.¹

Scope warning -- not universal scheduler correctness. The principal result is one fixed two-task witness: priorities 0 and 2, tick 5, and vTaskDelay(2) taking the non-wrapping quiet-resume path to wake 7. It is **not** a proof for every priority assignment, task population, scheduler state, delay argument, source branch, API call, or call sequence. In particular, it does not establish universal correctness of vTaskDelay, the five scheduler roots, or the whole FreeRTOS scheduler. The acceptance criteria for those stronger claims are recorded separately in UNIVERSAL_SCHEDULER_ACCEPTANCE.md.

Abstract

This report records a sealed milestone for one non-trivial FreeRTOS V6.1.1 scheduler path. A running priority-2 task calls the generated source function vTaskDelay'(2) at tick 5. The checked execution removes its generic list item from ready priority 2, writes wake key 7, inserts the item into the current delayed list, resumes through the quiet branch, returns Result(), and reaches the phase-indexed YieldPending endpoint representing task_delay_abs 2 p2_pre.

The earlier result was conditional on an endpoint and a byte-heap footprint. That condition has now been inhabited. The external build-and-check pipeline hash-locks a frozen portable ELF, an extracted and relink-validated layout ledger, and a generated CParser address configuration. Isabelle consumes the configuration as definitions; it does not prove an ELF-to-configuration correspondence theorem. Within the resulting artifact-specialised CParser/AutoCorres2 source semantics, six addressed C bases induce nine pairwise separated static xLIST regions; eight of those regions are the scheduler roots used by the P2 relation. Two fresh logical runtime TCB addresses, 0x00200000 and 0x00200100, are separated from each other and from all nine static regions. An explicit byte heap and generated global state satisfy the complete P2 precondition. Isabelle then discharges the previously conditional refinement theorem at that witness.

The repository also checks the literal source-monad chain

```
vListInitialise'  
-> vListInitialiseItem'  
-> vListInsertEnd'  
-> vListRemove'
```

with exactly three monadic binds, no theorem assumptions, and a normal Result() outcome. This theorem makes no claim about preservation of the sentinel's eight-byte over-read tail. The strict inventory now contains 13 source-to-abstract refinement theorems over eight distinct source operations, including two sequential-composition theorems.

The seal is deliberately narrower than whole-kernel correctness. It does not establish allocator or client boot reachability, execution of the context switch, compiler correctness, machine-code correctness, or full-scheduler verification.

1. Scope and statement discipline

The target is a single fixed positive-delay path, not all calls of vTaskDelay and not all scheduler behaviours. The abstract witness has two live tasks:

Symbol	Meaning
I	task P2_IDLE, priority 0
U	task P2_RUN, priority 2 and initially current
$G(t)$	generic list node embedded in the TCB of task t
a_0	abstract prestate p2_pre
a_1	abstract poststate task_delay_abs 2 p2_pre
D_*	concrete decoder frozen_p2_decode
R_*	generated_scheduler_roots
c_*	generated state frozen_p2_globals
$h(c)$	byte heap hrs_mem (t_hrs' c)
$E_\varphi(D,R,c,a)$	scheduler endpoint relation at phase φ
$F(D,R,H)$	source footprint on heap H

The operational judgment is

$$\begin{aligned}
& \text{RunsTo}(f, c, Q) \\
& \equiv \\
& \text{succeeds}(f, c) \\
& \wedge \\
& \forall r, t. \text{reaches}(f, c, r, t) \Rightarrow Q(r, t).
\end{aligned}$$

The positive `succeeds` conjunct excludes vacuous universal postconditions. The seal theorem therefore asserts both the existence of a suitable concrete preimage and a positive generated-source execution from that preimage.

The checked chain is:

```

frozen FreeRTOS V6.1.1 source slice
and externally validated address configuration
    |
    v
patched CParser / AutoCorres2 translation
    |
    v
generated monadic source semantics
    |
    v
Isabelle/HOL relation, heap, and refinement proofs
    |
    v
Isabelle kernel-checked theorem objects

```

The ELF builder, ledger generator, CParser, AutoCorres2, and Isabelle implementation belong to the stated toolchain boundary. The ELF-to-ledger-to-configuration link is externally regenerated and validated, not internalised as an Isabelle theorem. ChatGPT Pro, executable test harnesses, and any SMT search do not supply proof rules. The accepted theorems contain no `sorry`, `oops`, admitted axiom, oracle shortcut, skip-proof mode, or `quick_and_dirty=true` build.

2. The abstract P2 transition

The initial tick is 5 and the delay argument is 2, hence the modular wake time is

$$w = 5 +_{(32)} 2 = 7.$$

Because $7 \not\leq 5$, the execution takes the non-wrapping branch and inserts $G(U)$ into delayed list A. The abstract calculation checked in Isabelle is

$$\text{task_delay_abs}(2, a_0) = a_1.$$

The observable state change is exact:

Component	a_0 (p2_pre)	a_1
Live tasks	I, U	unchanged
Priority map	$I \mapsto 0, U \mapsto 2$	unchanged
Wake map	every task maps to None	$U \mapsto \text{Some}(7)$; all others unchanged
Ready 0	singleton $G(I)$, cursor at $G(I)$	unchanged
Ready 1	empty	empty
Ready 2	singleton $G(U)$, cursor at $G(U)$	empty
Ready 3	empty	empty
Delayed A	empty	singleton $G(U)$ with key 7 and no cursor
Delayed B	empty	empty
Pending / suspended	both empty	both empty
Tick / missed ticks	5 / 0	unchanged
Suspension depth	0	0
Missed-yield flag	false	false
Top-ready cache	2	2
Current task	$\text{Some}(U)$	$\text{Some}(U)$
Overflow count	0	0
Proof-port yield count	0	1

The endpoint is intentionally `YieldPending`. The priority-2 ready ring is already empty, while the cached top priority and current-task pointer have not yet been recomputed by a context switch. Thus a_1 satisfies the scheduler core invariant but not the stronger settled scheduler invariant. This is a phase fact, not a failed obligation.

3. Frozen artifact and addressed static roots

The frozen evidence tuple is identified by three primary digests:

Object	SHA-256
frozen_p2_layout.elf	DC830E50513384D712E0D1C68CB198EA656365F673D021C452D7D7EBD45C045A
layout_ledger.json	CA288A4CD2344BE979ADF09DBF0298C6715F196D64AE472D173304289C4F2C02

Object	SHA-256
generated address configuration	27F74768E1DB1C3F8DBFCFC85371075192BB7D2544ED324DC81B65A9A2911712

The local addressed-global CParser patch has digest 44160F97B133D0A66E515E505636D641907DC14811D43DA071EA15C706C8E604. It is applied to an upstream calculate_state.ML whose digest is EA51ECAA01947E53AD38A684B0E97ED360339363A75C6AE16DCFBFA7713562898; the staged patched file has digest FD244D8228E79EC3626A5CE312446CE49DF550970B758B68D3BBE953CAC8CFA9. The generated configuration causes the selected addressed globals to be emitted as definitions at the ledger addresses. It does not introduce logical axioms. The builder and generator verify the three-file correspondence outside Isabelle; the theorem layer starts from those generated definitions.

Exactly six C bases are mapped:

C base	Address	Extent
pxReadyTasksLists	0x00102020	80 bytes
xDelayedTaskList1	0x0010208c	20 bytes
xDelayedTaskList2	0x001020a0	20 bytes
xPendingReadyList	0x001020bc	20 bytes
xSuspendedTaskList	0x001020d4	20 bytes
xTasksWaitingTermination	0x001020e8	20 bytes

The ready-list base is an array of four 20-byte xLIST values. Therefore the six mapped bases induce the following nine static regions:

Static region	Address	Used by P2 relation?
ready[0]	0x00102020	yes
ready[1]	0x00102034	yes
ready[2]	0x00102048	yes
ready[3]	0x0010205c	yes
delayed A	0x0010208c	yes
delayed B	0x001020a0	yes
pending-ready	0x001020bc	yes
suspended	0x001020d4	yes
termination-wait	0x001020e8	no

This yields the audited cardinality chain

6\ mapped\ bases
 $\Rightarrow$
 9\ static\ xLIST\ regions
 $\Rightarrow$
 8\ P2\ relation\ roots.

The termination-wait list is outside the P2 abstract relation, but it remains inside the static separation theorem and the TCB-versus-static-region separation theorem. Omitting it from the geometry check would permit an unsound overlap even though the operation does not observe its list contents.

The diagnostic theorem `p2_source_footprint_delayed_alias_no_go` proves that if the two generated delayed-list roots alias, no P2 source-footprint witness can exist. This formally explains why a translation that leaves the two roots indistinguishable cannot close the milestone. Inside the artifact-specialised source semantics, the final construction does not assume their inequality: the addressed definitions and arithmetic geometry prove it. The claim that these definitions correspond to the frozen ELF is the separately validated evidence link described above.

Theorem Box 1 - static artifact geometry (frozen_addressed_xlist_geometry). Let A be the list of the nine addresses above and let $Reg_L(p)$ be the 20-byte list region at p . Isabelle proves

$distinct(A)$
 $\wedge \forall p \in A. guard(p)$
 $\wedge \forall p, q \in A. p \neq q \Rightarrow$
 $Reg_L(p) \cap Reg_L(q) = \emptyset.$

The theorem `frozen_p2_static_root_geometry` projects this result to the eight P2 roots.

4. Dynamic TCB geometry and source footprint

The frozen preimage uses two explicit runtime TCB pointers:

$tp_I = 0x00200000,$
 $tp_U = 0x00200100.$

They are fresh logical witness addresses. They are not claimed to be allocator results, ELF static objects, or states reached by the official boot path. Each TCB occupies 68 bytes. Its generic list item begins at offset 4 and its event list item begins at offset 24; each embedded item occupies 20 bytes. The checked interval facts include

$$\begin{aligned} &tp_I \neq tp_U, \\ &Reg_T(tp_I) \cap Reg_T(tp_U) = \emptyset, \\ &\forall tp \in tp_I, tp_U, \exists p \in A. \\ &Reg_T(tp) \cap Reg_L(p) = \emptyset, \\ &Reg_G(tp) \subseteq Reg_T(tp), \\ &Reg_E(tp) \subseteq Reg_T(tp), \\ &Reg_G(tp) \cap Reg_E(tp) = \emptyset. \end{aligned}$$

The theorem `frozen_p2_nonheap_geometry` packages decoder round trips, pointer guards, TCB separation, separation from all nine static regions, and the embedded Generic/Event geometry. Combining its facts with the eight-root static projection and maximum-key delayed sentinels proves

$$F(D_*, R_*, H_*),$$

where H_* is the explicitly constructed heap `frozen_p2_heap`.

The decoder maps I and U injectively to tp_I and tp_U , maps their embedded generic and event items to the corresponding abstract nodes, and supplies exact forward and reverse laws on the live set. No successful decode can silently identify the two tasks or confuse a generic item with an event item.

5. Explicit P2 heap and generated globals

The heap H_* starts from a zero heap, writes eight xLIST root values, and writes the two live generic list items. Its abstract views are:

$$\begin{aligned} R_0 &\mapsto [G(I)]_-(\mapsto G(I)), \\ R_1 &\mapsto \varepsilon, \\ R_2 &\mapsto [G(U)]_-(\mapsto G(U)), \\ R_3 &\mapsto \varepsilon, \\ D_A &\mapsto \varepsilon, \\ D_B &\mapsto \varepsilon, \\ P &\mapsto \varepsilon, \\ S &\mapsto \varepsilon. \end{aligned}$$

Both delayed-list sentinels contain the maximum key required by ordered insertion. The singleton ready-list roots contain consistent count, cursor, next, previous, owner, and container fields. The proofs read these values back through the generated heap selectors; the abstract relation is not postulated.

The state c_* sets the generated scheduler fields to the P2 values: delayed A is current, delayed B is overflow, tick is 5, scheduler suspension and missed ticks are zero, missed yield is false, top-ready cache is 2, live-task count is 2, overflow count is zero, yield count is zero, `pxCurrentTCB` decodes to U , the scheduler-running word is 1, critical depth is zero, and interrupts are enabled.

Theorem Box 2 - concrete endpoint (`frozen_p2_endpoint`). The explicit decoder, generated roots, heap, and globals satisfy

$$E_(\text{StableRunning})(D_*, R_*, c_*, a_0).$$

Together with the footprint theorem, this proves the non-vacuity statement

$$\begin{aligned} &\exists D R c. E_(\text{StableRunning})(D, R, c, a_0) \\ &\wedge F(D, R, h(c)). \end{aligned}$$

The checked theorem is `frozen_p2_preimage_nonempty`. It closes the earlier open preimage obligation without asserting that c_* is reachable from reset or official scheduler initialisation.

6. Exact generated-source execution and refinement

For a generic state satisfying the endpoint and footprint, the artifact-specialised generated body has an exact-state theorem. Let

$$\begin{aligned} H_0 &\equiv h(c), \\ H_r &\equiv \text{raw_remove_concrete_heap}(H_0, p_U), \\ H_k &\equiv \text{scheduler_generic_item_key_heap}(H_r, tp_U, 7), \\ H_f &\equiv \text{raw_ordered_insert_empty_heap}(H_k, D_A, p_U). \end{aligned}$$

The final generated state is

$$S(c, D, R) = \text{Yield}(\text{ResumeQuiet}(\text{Mem}(H_f, \text{Suspend}(c)))).$$

The theorem `scheduler_vTaskDelay_2_p2_exact_state` proves positive execution and equality of every admitted final state with $S(c,D,R)$. The pure theorem `p2_remove_wake_insert_lists_rel` separately proves the representation of all eight P2 list roots after removal, key write, and insertion. The assembly theorem `scheduler_vTaskDelay_2_p2_refines_task_delay_abs` states

$$\begin{aligned} &E_(\text{StableRunning})(D,R,c,a_0) \wedge F(D,R,h(c)) \\ &\Rightarrow \text{RunsTo}(\text{vTaskDelay}'(2),c, \\ &\lambda(r,t). r=\text{Result}() \wedge \\ &E_(\text{YieldPending})(D,R,t,\text{task_delay_abs}(2,a_0))). \end{aligned}$$

The new preimage theorem supplies this implication's antecedent. Isabelle checks the specialised theorem

$$\begin{aligned} &\text{RunsTo}(\text{vTaskDelay}'(2),c_*, \\ &\lambda(r,t). r=\text{Result}() \wedge \\ &E_(\text{YieldPending})(D_*,R_*,t,\text{task_delay_abs}(2,a_0))). \end{aligned}$$

Theorem Box 3 - frozen-artifact seal (`frozen_p2_artifact_bound_seal`). Isabelle proves

$$\begin{aligned} &\exists D R c. E_(\text{StableRunning})(D,R,c,a_0) \\ &\wedge F(D,R,h(c)) \\ &\wedge \text{RunsTo}(\text{vTaskDelay}'(2),c, \\ &\lambda(r,t). r=\text{Result}() \wedge \\ &E_(\text{YieldPending})(D,R,t,\text{task_delay_abs}(2,a_0))). \end{aligned}$$

The witness is artifact-bound for the static scheduler-root layout and logical for the two runtime TCB allocations. That qualification is part of the seal, not a caveat added after the theorem.

7. Literal four-call source-monad hard gate

The independent list hard gate is the literal generated-function composition

$$\begin{aligned} C_4 \equiv & \\ &\text{bind}(\text{vListInitialise}'(L),\lambda_ \\ &\text{bind}(\text{vListInitialiseItem}'(P),\lambda_ \\ &\text{bind}(\text{vListInsertEnd}'(L,P),\lambda_ \\ &\text{vListRemove}'(P))). \end{aligned}$$

The definition `raw_initialise_insert_remove_needle'` contains exactly three `bind` nodes. It uses the fixed list address `0x00001000`, item address `0x00002000`, and sentinel address `0x00001008`. The theorem `raw_vListInitialise_insert_end_remove_refines` has no assumptions and proves

$$\begin{aligned} &\text{RunsTo}(C_4,s, \\ &\lambda(r,t). r=\text{Result}() \wedge \exists k. \\ &\text{raw_xlist_rel}(h(t),L,\text{list_remove_abs} \\ &(P,\text{list_insert_end_abs}(P,k,\text{empty}(\text{keys}))))). \end{aligned}$$

The round-trip simplifies to an empty list whose total key map is updated at P . A key-congruence lemma then yields the clean corollary `raw_vListInitialise_insert_end_remove_empty_refines`, which returns to `raw_empty_abs keys` because non-live keys are observationally irrelevant to the empty relation.

This is a source-monad composition theorem assembled from already checked exact and general contracts at their actual intermediate states. It does not equate the insert result with an independently constructed removal prestate. It also does not claim that the full eight-byte sentinel over-read tail is preserved by the four-call sequence. Tail framing is proved elsewhere for narrower source contracts and is deliberately absent from this hard-gate statement.

8. Sealed checker evidence and inventory

The final portable runs relevant to this seal are:

Proof layer	Run identifier	Time	Result
Address-bound scheduler parse	20260801Tseal-scheduler-parse-01-portable	23.566 s	exit 0; QAD false
Delayed-root alias no-go	20260801Tseal-p2-layout-no-go-01-portable	33.162 s	exit 0; QAD false
Nine-region static geometry	20260801Tseal-p2-static-nine-01-portable	27.701 s	exit 0; QAD false
TCB separation from all nine regions	20260801Tseal-p2-dynamic-all-nine-01-portable	27.467 s	exit 0; QAD false
Explicit P2 preimage and refinement seal	20260801Tseal-p2-preimage-06-parenthesised-seal	120.854 s	exit 0; QAD false
Literal four-call list composition	20260801Tseal-list-four-call-01-portable	32.338 s	exit 0; QAD false
Capstone theorem-object assumption audit	20260801Tseal-assumption-audit-01-portable	137.387 s	exit 0; QAD false

Every status file records the same frozen ELF, ledger, and generated-address-configuration digests shown in Section 3. The final stdout digests, in table order, are:

1. 36AA4AE5EDA156D123DAAC5A53A5DB76CC5C817B8B7B8F9378163E9C94A3363D;
2. 286AF01D88A5007CECBD1860739AE206F1A5083220501E9088054B1A412A1DF9;
3. BB442ECA20CE4753278CC884B79E403BC1EC0B82DAE7843A5AC5B68F0B2130C5;
4. BB442ECA20CE4753278CC884B79E403BC1EC0B82DAE7843A5AC5B68F0B2130C5;
5. 1E51B030BD7D73DD4491CDA05712ED099BEAD1D340F64C96242CD7E168B902CC;
6. B71D9682664FF9E68DBB2F93EE195DF4AB50BFD9CA6A2B62CE444FC10D6F427C;
7. 9186E921B5881C9412AE3FEB98CBAECF92A1E2E734F3A1344D9AEBF4CB88B1C3.

The repeated static/dynamic stdout digest is expected because the dynamic session reuses the same successful dependency closure and produces the same bounded log text. Hash equality is a provenance observation, not a substitute for the exit status or theorem object.

The final leaf audit inspects five capstone theorem objects: the two four-call results and the frozen-P2 preimage, refinement, and seal. Every object has empty `Thm.premis_of` and `Thm.hyps_of`; its complete proposition contains no `Pure.imp` or `HOL.implies`; and both `Thm.shyps_of` and `Thm.extra_shyps` have cardinality zero. Consequently none retains a function-address, addressed-data-global, or generated G/S locale premise. This statement is about theorem assumptions only and does not turn the external ELF-to-configuration evidence check into an Isabelle theorem.

The strict source-to-abstract inventory now records:

Measure	Sealed count	Interpretation
Refinement theorems	13	individual strict source-to-abstract entries, including the artifact P2 result and four-call chain
Distinct source operations	8	compositions do not inflate the operation count
Sequential compositions	2	remove-then-insert-end and initialise/initialise-item/insert-end/remove

The artifact-bound P2 theorem strengthens the already counted `vTaskDelay` operation, so it adds a refinement theorem without adding a ninth operation. The four-call theorem is a new composition over already counted list operations and likewise adds no distinct operation.

9. Historical progression and corrected ledger

The development passed through five materially different stages:

1. The abstract P2 calculation and phase-indexed endpoint were proved first.
2. The generated `vTaskDelay'` (2) body and all-eight-roots postrelation yielded a conditional source-to-abstract theorem. At that time the accurate status was: source refinement green, frozen preimage open.
3. The delayed-root alias no-go isolated the addressed-global obstruction. The external builder/generator checks then linked one frozen ELF/ledger tuple to a configuration that definitionally fixed the six relevant C bases in `CParser`.
4. Static and dynamic geometry proved the exact $6 \rightarrow 9 \rightarrow 8$ accounting, introduced two fresh logical TCB witnesses, and separated their 68-byte regions from all nine static list regions.
5. The explicit heap and globals discharged the conditional antecedent, and the independent literal four-call theorem closed the source-monad composition hard gate.

Earlier reports used the phrase "eight roots" for the P2 relation and could be read as saying that only eight static list regions mattered. The final ledger corrects that ambiguity: there are six mapped bases, nine static `xLIST` regions, and eight relation roots. The ninth region, termination-wait, is excluded only from the abstract P2 list relation; it remains included in static and dynamic non-alias proofs.

Historical red runs document proof search and do not belong to the sealed evidence set. The seven final run identifiers in Section 8 supersede intermediate green runs for the artifact binding, geometry, preimage, four-call, and assumption-audit claims. The earlier conditional theorems remain useful reusable general results; they are not retracted, but their premises are now instantiated for this witness.

10. Tooling assessment and trust boundary

No additional symbolic executor, verification-condition generator, or SMT backend is required to finish this sealed P2 milestone. The operational work already uses `CParser/AutoCorres2` generated monadic semantics and Isabelle proof/VCG infrastructure. The hard parts were representation alignment, byte-heap readback, alias geometry, frame reasoning, and phase-correct relation assembly. A second generic executor would not replace those invariants.

Future scheduler-wide work could benefit from a heap-aware path and VC generator that proposes frame obligations, plus an SMT solver that proposes finite address arithmetic or disjointness facts. Such output should remain untrusted candidate evidence and be replayed by Isabelle. The useful architecture is therefore

```

path / heap-VC generator -> candidate obligations
SMT backend              -> candidate arithmetic facts
Isabelle replay          -> accepted theorem objects

```

The following claims remain explicitly outside this report:

- allocator correctness or allocation of the two logical TCB witnesses;
- reachability of `c_*` from reset, task creation, or scheduler initialisation;
- execution and correctness of `vTaskSwitchContext` after `YieldPending`;
- compiler, assembler, linker, loader, or machine-code correctness;
- correspondence between every C execution and a target-processor execution;
- arbitrary task sets, priorities, delay values, wrap branches, or callers;
- whole-list-library or whole-scheduler functional correctness.

These exclusions do not weaken the stated theorem. They delimit which antecedents and semantic layers the theorem actually contains.

Conclusion

The scoped blind-reconstruction milestone is sealed. The literal alias-sensitive source-monad chain `vListInitialise' -> vListInitialiseItem' -> vListInsertEnd' -> vListRemove'` is kernel-green without assumptions and contains exactly three binds. In the artifact-specialised `CParser/AutoCorres2` source semantics, the generated configuration definitionally fixes six addressed-data bases, and Isabelle proves the resulting nine static list regions distinct, guarded, and pairwise disjoint. The external builder/generator validates that configuration against the hash-locked frozen ELF and ledger; this correspondence is not itself an Isabelle theorem. An artifact-root-bound P2 state exists using fresh logical TCB witnesses at `0x00200000` and `0x00200100`. From that state, the generated FreeRTOS V6.1.1 `vTaskDelay'(2)` source semantics returns `Result()` and refines `task_delay_abs 2 p2_pre` at `YieldPending`.

The strongest accurate one-line status is:

Sealed frozen-artifact P2 milestone: non-vacuous generated-source refinement is green; boot/allocator reachability, context-switch execution, compiler and machine-code correctness, and full-scheduler verification are outside scope.

Appendix A. Compact proof ledger

The seal can be compressed to five checked layers:

Layer	Sealed mathematical fact
Static roots	$Geometry_9(A) \wedge Projection_8(A, R_*)$
Logical TCBs	$TCBGeometry(D_*, tp_l, tp_u) \wedge SeparateFrom_9(tp_l, tp_u, A)$
Concrete witness	$E_ (StableRunning)(D_R, c, a_0) \wedge F(D_R, h(c))$
Generic refinement	$E_ (StableRunning)(D, R, c, a_0) \wedge F(D, R, h(c)) \Rightarrow RunsTo(vTaskDelay'(2), c, Q_ (D, R))$
Final seal	$\exists D R c. E_ (StableRunning)(D, R, c, a_0) \wedge F(D, R, h(c)) \wedge RunsTo(vTaskDelay'(2), c, Q_ (D, R))$

Here $Q_ (D, R)(r, t)$ abbreviates $r = Result() \wedge E_ (YieldPending)(D, R, t, task_delay_abs(2, a_0))$.

Appendix B. Interpretation checklist

- "Frozen artifact / 6 / 9 / 8" means the three hashes fix the external evidence tuple; six addressed C bases expand to nine static xLIST regions, of which eight are P2 roots.
- "Artifact-bound P2" means external checks link the ELF and ledger to the static-root configuration consumed by Isabelle; the two runtime TCB addresses remain logical witnesses.
- "Preimage / generated source / YieldPending" means explicit D_R, c_* inhabit the predicates, execution succeeds in the specialised source semantics, and no context-switch execution is claimed.
- "Exactly three binds / no tail8" characterises the literal four-call syntax and its deliberately limited frame claim.
- "No capstone assumptions" means the five audited theorem objects have no rule premises, context hypotheses, proposition implications, or sort hypotheses.
- "13 / 8 / 2" means 13 refinement theorems, eight distinct source operations, and two sequential compositions.
- "Sealed" refers only to this stated milestone and its final evidence set.

1. No external proof, source artifact, theorem, or generated obligation from ChatGPT Pro was admitted. Its contribution was critical review of scope and prose. Isabelle theorem objects and the recorded artifact/run evidence remain authoritative.